Finite State Machine (FSM) Modeling

RTL Design Fundamentals

October 12, 2026

Contents

1 Introduction **2**

2 Three-Process FSM Modeling **2**

3 Example 1: Moore FSM **2**
3.1 Verilog Code 2
3.2 Working 3

4 Example 2: Mealy FSM **4**
4.1 Verilog Code 4
4.2 Working 5

5 Moore vs. Mealy FSM **5**

6 FSM Coding Guidelines **5**

7 Summary **6**

1 Introduction

A Finite State Machine (FSM) is a sequential circuit that operates through a finite number of states.

The next state depends on the current state and the input conditions. Outputs depend on the current state, or on the current state and inputs.

FSMs are widely used in controllers, communication protocols, traffic lights, and processor control logic.

The two common types are:

- **Moore FSM:** Outputs depend only on the current state.
- **Mealy FSM:** Outputs depend on the current state and current inputs.

2 Three-Process FSM Modeling

A common RTL coding style separates the FSM into three procedural blocks:

1. State register: updates the current state on the active clock edge.
2. Next-state logic: determines the next state using the current state and inputs.
3. Output logic: generates outputs based on the FSM requirements.

This separation improves readability and simplifies verification and debugging.

3 Example 1: Moore FSM

Consider a controller with three states:

- `IDLE`: Waits for a start signal.
- `ACTIVE`: Performs an operation.
- `DONE`: Indicates completion.

The output `done` depends only on the current state.

3.1 Verilog Code

```
module moore_fsm (
    input  wire clk,
    input  wire rst,
    input  wire start,
    input  wire finish,
    output reg  done
);

    localparam IDLE   = 2'b00,
               ACTIVE = 2'b01,
               DONE   = 2'b10;
```

```
12
13      reg [1:0] state, next_state;
14
15      // 1. State register
16      always @(posedge clk) begin
17          if (rst)
18              state <= IDLE;
19          else
20              state <= next_state;
21      end
22
23      // 2. Next-state logic
24      always @(*) begin
25          next_state = state;
26
27          case (state)
28              IDLE:
29                  if (start)
30                      next_state = ACTIVE;
31
32              ACTIVE:
33                  if (finish)
34                      next_state = DONE;
35
36              DONE:
37                  next_state = IDLE;
38
39              default:
40                  next_state = IDLE;
41          endcase
42      end
43
44      // 3. Output logic
45      always @(*) begin
46          done = 1'b0;
47
48          case (state)
49              DONE: done = 1'b1;
50              default: done = 1'b0;
51          endcase
52      end
53
54  endmodule
```

3.2 Working

1. The FSM starts in `IDLE` after reset.
2. When `start = 1`, it moves to `ACTIVE` at the next rising clock edge.
3. When `finish = 1`, it moves to `DONE` at the next rising clock edge.

4. In `DONE`, the output `done` becomes 1.
5. The FSM returns to `IDLE`.

4 Example 2: Mealy FSM

In a Mealy FSM, the output depends on the current state and an input.

Consider a controller that asserts `ready` when the FSM is in `WAIT` and the input `valid` is high.

4.1 Verilog Code

```
module mealy_fsm (
    input  wire clk,
    input  wire rst,
    input  wire start,
    input  wire valid,
    output reg  ready
);

    localparam IDLE = 1'b0,
               WAIT = 1'b1;

    reg state, next_state;

    // 1. State register
    always @(posedge clk) begin
        if (rst)
            state <= IDLE;
        else
            state <= next_state;
    end

    // 2. Next-state logic
    always @(*) begin
        next_state = state;

        case (state)
            IDLE:
                if (start)
                    next_state = WAIT;

            WAIT:
                if (valid)
                    next_state = IDLE;

            default:
                next_state = IDLE;
        endcase
    end
```

```
39
40      // 3. Mealy output logic
41      always @(*) begin
42          ready = 1'b0;
43
44          if ((state == WAIT) && valid)
45              ready = 1'b1;
46      end
47
48  endmodule
```

4.2 Working

The output `ready` becomes 1 when both conditions are true:

$$\text{state} = \text{WAIT} \quad \text{and} \quad \mathit{valid} = 1$$

Unlike the Moore example, the output can change immediately when `valid` changes, even if the state does not change.

5 Moore vs. Mealy FSM

Feature	**Moore FSM**	**Mealy FSM**
Output depends on	Current state	Current state and inputs
Output response	Usually changes after a state update	Can change when inputs change
Output stability	Often easier to keep stable	May respond to input glitches
State count	May require more states	May require fewer states

6 FSM Coding Guidelines

- Use named `localparam` constants for state encodings.
- Use nonblocking assignments in the clocked state-register block.
- Use blocking assignments in combinational next-state and output logic.
- Assign default values to combinational signals to avoid unintended latches.
- Include a suitable `default` branch for invalid or unrecognized states.
- Use a consistent reset strategy.
- Verify state transitions and outputs through simulation.
- Review synthesis reports to confirm the intended FSM implementation.

7 Summary

- An FSM consists of states, transitions, inputs, and outputs.
- A Moore FSM generates outputs from the current state.
- A Mealy FSM generates outputs from the current state and inputs.
- A three-process coding style separates state storage, transition logic, and output logic.
- Proper defaults, reset logic, and assignment styles improve RTL quality.

Key takeaway: FSM coding should clearly separate sequential state updates from combinational next-state and output logic.